

Compilador Web Interactivo

YALex + YAPar — Documentación Técnica Completa

Curso: Lenguajes Formales y de Programación

Stack: C++17 · Python/FastAPI · React/TypeScript

Junio 2026

Resumen del Proyecto

Este proyecto implementa un **IDE web completo** para el análisis léxico y sintáctico de lenguajes formales. El núcleo está escrito en C++17 y construye autómatas finitos, tablas de parsing SLR/LALR/LL(1) y árboles sintácticos desde cero. Una API FastAPI expone el motor C++ al mundo web, y un frontend React permite editar, analizar y visualizar todo en tiempo real desde el navegador.

Tabla de Contenidos

1. ¿Qué es este proyecto? — Visión general
2. Arquitectura del sistema
3. Módulo YALex — Análisis Léxico
 - 3.1 ¿Qué es el análisis léxico?
 - 3.2 Lectura del archivo .yalex
 - 3.3 Motor de expresiones regulares
 - 3.4 Construcción del NFA (Thompson)
 - 3.5 De NFA a DFA (Subconjuntos)
 - 3.6 Minimización del DFA (Hopcroft)
 - 3.7 Tokenizador final
4. Módulo YAPar — Análisis Sintáctico
 - 4.1 ¿Qué es el análisis sintáctico?
 - 4.2 Carga y validación de la gramática
 - 4.3 Conjuntos FIRST y FOLLOW
 - 4.4 Autómata LR(0)
 - 4.5 Tabla SLR(1)
 - 4.6 Tabla LALR(1)
 - 4.7 Tabla LL(1)
 - 4.8 Parser (análisis bottom-up)
 - 4.9 Resolución de conflictos
5. API REST (Python / FastAPI)
6. Frontend (React / TypeScript)
7. Flujo completo de datos — ejemplo paso a paso
8. Formatos de entrada: .yalex y .yapar
9. Resumen y conclusiones

1. ¿Qué es este proyecto?

Este proyecto es un **compilador web interactivo**. Su propósito es permitir que alguien defina su propio lenguaje de programación (o cualquier lenguaje formal) a través de dos archivos de especificación, y luego analice texto de entrada para ver si es válido en ese lenguaje, qué tokens contiene y cómo se parsea.

Para entenderlo con una analogía: si quisieras crear el lenguaje **Q'eqchi'Script**, primero describirías cuáles son sus palabras clave y patrones de texto (análisis léxico) y luego definirías las reglas gramaticales de cómo se combinan (análisis sintáctico). Este proyecto hace exactamente eso de forma automática.

¿Para qué sirve concretamente?

Dado un archivo **.yalex** (especificación léxica) y un archivo **.yapar** (especificación gramatical), el sistema puede:

- Reconocer automáticamente los tokens (palabras) de cualquier texto de entrada.
- Construir el autómata finito determinista (DFA) que reconoce esos tokens.
- Construir las tablas de parsing (SLR, LALR, LL1) para la gramática.
- Validar si un texto respeta la gramática y generar el árbol sintáctico.
- Visualizar todo en un IDE web con editor de código, consola de errores y diagramas.

Soporte de lenguajes especiales

El proyecto también incluye soporte experimental para lenguajes creativos como **Q'eqchi'** (lengua maya de Guatemala), **MessiScript**, **COW** y **OK COMPUTER**, con detección automática del tipo de archivo.

2. Arquitectura del Sistema

El sistema tiene **tres capas** bien diferenciadas que se comunican entre sí:

Capa	Tecnología	Responsabilidad
Frontend	React 18 + TypeScript Vite + Monaco Editor	IDE web: editores, visualizaciones, consola
API	Python 3 + FastAPI Pydantic + CORS	Recibe JSON del frontend, invoca los binarios C++
Core	C++17 + CMake nlohmann/json	Toda la lógica: regex, NFA, DFA, gramática, tablas, parser

Flujo general de comunicación:

El usuario escribe o sube archivos en el **frontend**. Al hacer clic en "Ejecutar", el navegador envía una petición HTTP POST a la **API** en Python. La API guarda los contenidos en archivos temporales y ejecuta el binario C++ correspondiente (yalex_cli o yapar_cli) como subproceso. El binario devuelve un JSON por stdout que la API reenvía al frontend. El frontend parsea el JSON y actualiza todas las pestañas de visualización.

// Flujo de datos

```
[Navegador / React] | | HTTP POST /yalex/analyze | HTTP POST /yapar/analyze v [FastAPI  
Server - puerto 8000] | | subprocess.run(yalex_cli, ...) | subprocess.run(yapar_cli,  
...) v [Binario C++ compilado] | | stdout: JSON con tokens/DFA/tablas/arbol v [FastAPI  
-> Frontend -> Visualizaciones]
```

3. Módulo YALex — Análisis Léxico

3.1 ¿Qué es el análisis léxico?

El análisis léxico es el primer paso de un compilador. Su trabajo es leer una secuencia de caracteres y agruparlos en **tokens**: las unidades mínimas con significado. Por ejemplo, al analizar la expresión `x := 3 + y`, un analizador léxico produce los tokens: `ID(x)`, `ASSIGN`, `INT(3)`, `PLUS`, `ID(y)`.

Para saber qué es un token, se usan **expresiones regulares**. Cada token se define como un patrón regex. Por ejemplo: `ID = [a-z][a-z0-9]*` significa que un identificador empieza con minúscula y puede seguir con letras o dígitos.

3.2 Lectura del archivo .yalex

El archivo **.yalex** es la especificación léxica. Tiene dos partes: primero se definen **macros** (nombres para patrones reutilizables) y luego se definen las **reglas** que asocian cada patrón con un nombre de token.

// Ejemplo de archivo .yalex

```
let digit = ["0"-"9"] let letter = ["a"-"z"] | ["A"-"Z"] let id = letter (letter |
digit)* rule tokens = digit+ { INT } | id { ID } | "+" { PLUS } | "-" { MINUS } | " " |
"\t" { ignore }
```

El módulo `yalex_reader.cpp` (899 líneas) lee este formato y también soporta variantes como `.yal`, `.lex` y `.l`. Extrae las macros y las reglas, y las pasa al motor de expresiones regulares.

3.3 Motor de Expresiones Regulares

El archivo `regex_engine.cpp` convierte cada expresión regular en una forma que la máquina puede procesar. Lo hace en **tres pasos**:

- **Paso 1 — Expansión de macros:** reemplaza las referencias como `{digit}` por su definición real.
- **Paso 2 — Expansión de clases de caracteres:** convierte `[a-z]` en `(a|b|c|...|z)`.
- **Paso 3 — Conversión a postfix (Shunting-Yard):** reorganiza los operadores para que sea fácil construir el autómata.

// regex_engine.cpp — precedencia de operadores

```
// El algoritmo Shunting-Yard convierte infijo a postfijo: // Entrada (infijo): a | b *
c // Salida (postfijo): a b c * | // // Prioridades de operadores: // * + ? (unarios,
más alta) -> precedencia 3 // concat (implícito) -> precedencia 2 // | (alternativa) ->
precedencia 1 int RegexParser::precedence(RegexTokenType type) { switch (type) { case
KLEENE_STAR: case PLUS: case OPTIONAL: return 3; case CONCAT: return 2; case UNION:
return 1; default: return 0; } }
```

3.4 Construcción del NFA (Algoritmo de Thompson)

Una vez que la regex está en notación postfix, se construye un **NFA (Non-deterministic Finite Automaton)** usando el **algoritmo de Thompson**. La idea es simple: cada operador regex se convierte en un pequeño fragmento de autómatas, y los fragmentos se ensamblan:

Regex	Fragmento NFA
Literal 'a'	Estado inicial → (con 'a') → Estado final
A B (unión)	Nuevo inicio con ϵ a A y B; sus finales van a nuevo fin con ϵ
AB (concat)	El final de A se convierte en el inicio de B
A* (Kleene)	Nuevo inicio y fin; ϵ desde inicio a A-inicio y a nuevo fin; ϵ desde A-fin a A-inicio y a nuevo fin
A+ (plus)	igual que A*, pero sin la ϵ directa inicio→fin
A? (opcional)	igual que A*, pero sin el bucle interno

3.5 De NFA a DFA — Construcción de Subconjuntos

Un NFA puede estar en varios estados a la vez (no-determinismo). Para poder simularlo eficientemente, se convierte en un **DFA (Deterministic Finite Automaton)** usando el algoritmo de **construcción de subconjuntos**:

- Se calcula la ϵ -**clausura** del estado inicial del NFA (todos los estados alcanzables sin consumir entrada).
- Ese conjunto de estados del NFA se convierte en un **estado del DFA**.
- Para cada símbolo posible, se calculan los estados NFA alcanzables y su ϵ -clausura → nuevo estado DFA.
- Se repite hasta no haber estados nuevos.

3.6 Minimización del DFA — Algoritmo de Hopcroft

La construcción de subconjuntos puede generar estados DFA redundantes. El **algoritmo de Hopcroft** los minimiza agrupando estados equivalentes (estados que se comportan igual para toda entrada futura):

- Se parte de dos grupos: estados de aceptación y no-aceptación.
- Iterativamente se subdividen grupos: si dentro de un grupo dos estados transicionan a grupos distintos para algún símbolo, se separan.
- Cuando no hay más subdivisiones, cada grupo se convierte en un estado del DFA mínimo.

3.7 Tokenizador Final

El archivo tokenizer.cpp orquesta todo el proceso léxico. Combina los DFAs de todas las reglas del .yalex en uno solo (con prioridad: el token declarado primero gana en caso de empate). Luego recorre el texto de entrada y produce la lista de tokens usando simulación del DFA.

El resultado es un JSON con: lista de tokens encontrados, la representación del DFA (estados, transiciones), y un grafo en formato Graphviz DOT para visualizar el autómata.

4. Módulo YAPar — Análisis Sintáctico

4.1 ¿Qué es el análisis sintáctico?

El análisis sintáctico (parsing) toma la lista de tokens producida por el analizador léxico y verifica que su secuencia respeta la **gramática** del lenguaje. Si es válida, produce un **árbol sintáctico** que representa la estructura jerárquica del programa.

Las gramáticas se expresan en **BNF (Backus-Naur Form)**. Por ejemplo, una gramática simple para expresiones aritméticas:

```
// Ejemplo de archivo .yapar
```

```
%token INT PLUS TIMES LPAREN RPAREN %start expr %%  
expr : expr PLUS term | term ; term  
: term TIMES factor | factor ; factor : LPAREN expr RPAREN | INT ;
```

4.2 Carga y Validación de la Gramática

El archivo grammar.cpp (603 líneas) lee el archivo .yapar. El formato soporta: declaración de tokens con %token, símbolo inicial con %start, asociatividad/precedencia con %left/%right/%nonassoc, y las reglas de producción tras el separador %%.

Tras cargar, realiza **validaciones**: verifica que exista el separador %%, que haya al menos una producción, que el símbolo inicial esté definido, y que todos los no-terminales usados tengan sus producciones.

Finalmente **augmenta la gramática** añadiendo una producción $S' \rightarrow S$ que es necesaria para construir las tablas SLR/LALR correctamente.

```
// grammar.cpp - Gramática aumentada
```

```
// Aumentar la gramática:  $S' \rightarrow S$  // Es imprescindible para que el estado ACCEPT sea  
único if (!startSymbol_.name.empty() && !productions_.empty()) { std::string augName =  
startSymbol_.name + "''"; Production aug; aug.lhs = Symbol::nonTerminal(augName);  
aug.rhs = { startSymbol_ }; productions_.insert(productions_.begin(), aug);  
startSymbol_ = Symbol::nonTerminal(augName); }
```

4.3 Conjuntos FIRST y FOLLOW

Para construir las tablas de parsing, primero hay que calcular dos conjuntos para cada símbolo de la gramática:

- **FIRST(A)**: el conjunto de terminales con los que puede comenzar cualquier cadena derivable de A. Si A puede derivar ϵ , también incluye ϵ .
- **FOLLOW(A)**: el conjunto de terminales que pueden aparecer inmediatamente después de A en alguna forma sentencial. El símbolo inicial siempre tiene \$ (fin de entrada) en su FOLLOW.

Ambos se calculan con un algoritmo de **punto fijo**: se inicializan vacíos y se iteran las reglas hasta que no haya cambios.

```
// grammar.cpp - Cálculo de FIRST (punto fijo)
```



```
// FIRST – algoritmo de punto fijo // Para cada producción  $A \rightarrow X_1 X_2 \dots X_n$ : // Agrega
FIRST( $X_1$ ) -  $\{\epsilon\}$  a FIRST( $A$ ) // Si  $\epsilon \in \text{FIRST}(X_1)$ : también agrega FIRST( $X_2$ ) -  $\{\epsilon\}$  // ... y
así hasta  $X_i$  donde  $\epsilon \notin \text{FIRST}(X_i)$  // Si todos  $X_i$  tienen  $\epsilon$ : agrega  $\epsilon$  a FIRST( $A$ ) bool
changed = true; while (changed) { changed = false; for (const auto& prod :
productions_) { bool allHaveEpsilon = true; for (const auto& Xi : prod.rhs) { for
(const auto& f : first_[Xi.name]) if (f != EPSILON) changed |=
first_[A].insert(f).second; if (!first_[Xi.name].count(EPSILON)) { allHaveEpsilon =
false; break; } } if (allHaveEpsilon) changed |= first_[A].insert(EPSILON).second; } }
```

4.4 Autómata LR(0)

Un **ítem LR(0)** es una producción con un punto ($\bullet$) que indica hasta dónde se ha leído. Por ejemplo, $E \rightarrow E \bullet + T$ significa que ya se leyó E y se espera ver $+$ luego T .

El **autómata LR(0)** tiene un estado por cada conjunto de ítems. Se construye con dos operaciones:

- **closure(I)**: si en el estado hay un ítem $A \rightarrow \alpha \bullet B \beta$, se añaden también los ítems $B \rightarrow \bullet \gamma$ para todas las producciones de B .
- **goto(I, X)**: nuevo estado formado por avanzar el punto sobre X en todos los ítems de I donde el símbolo tras el punto es X , y aplicar closure.

// lr0_automaton.cpp – función closure

```
// closure(I): cierra un conjunto de ítems LR(0) // Si  $A \rightarrow \alpha \bullet B \beta$  está en  $I$  y  $B$  es
no-terminal, // añadimos todos los ítems  $B \rightarrow \bullet \gamma$  std::set<LR0Item>
LR0Automaton::closure(const std::set<LR0Item>& items) { std::set<LR0Item>
result = items; std::queue<LR0Item> worklist; for (const auto& item : items)
worklist.push(item); while (!worklist.empty()) { LR0Item current = worklist.front();
worklist.pop(); std::string B = symbolAfterDot(current); if (B.empty()) continue; //
punto al final // Agregar todos los ítems  $B \rightarrow \bullet \gamma$  for (int prodId :
grammar->getProductionsFor(B)) { LR0Item newItem{prodId, 0}; if
(!result.count(newItem)) { result.insert(newItem); worklist.push(newItem); } } } return
result; }
```

4.5 Tabla SLR(1)

La tabla **SLR(1)** (Simple LR) usa el autómata LR(0) + los conjuntos FOLLOW para decidir qué acción tomar en cada estado según el siguiente token:

Situación	Acción
El ítem es $A \rightarrow \alpha \bullet a \beta$ (a es terminal)	SHIFT: pasar al estado goto(s, a)
El ítem es $A \rightarrow \alpha \bullet$ (punto al final), $a \in \text{FOLLOW}(A)$	REDUCE: aplicar $A \rightarrow \alpha$
El ítem es $S' \rightarrow S \bullet$ (gramática aumentada)	ACCEPT
goto(s, A) existe para no-terminal A	GOTO: transición al nuevo estado

4.6 Tabla LALR(1)

La tabla **LALR(1)** (Look-Ahead LR) es más potente que SLR. La diferencia clave es que usa **lookaheads específicos por ítem** en vez del FOLLOW completo del no-terminal. Esto reduce los conflictos shift/reduce en gramáticas donde SLR sería ambigua.

El proceso: se construye el autómata LR(0) normal, y luego se propagan los lookaheads desde el estado inicial hasta todos los ítems, usando la operación de closure con lookaheads (ítems LR(1)).

4.7 Tabla LL(1)

La tabla **LL(1)** es para parsing **top-down** (de izquierda a derecha, derivando el símbolo más a la izquierda). Para cada par (no-terminal, terminal), dice qué producción usar:

- Si $A \rightarrow \alpha$ y $a \in \text{FIRST}(\alpha)$: añadir $A \rightarrow \alpha$ en la celda $[A, a]$.
- Si $A \rightarrow \alpha$ y $\epsilon \in \text{FIRST}(\alpha)$ y $a \in \text{FOLLOW}(A)$: añadir $A \rightarrow \alpha$ en $[A, a]$.

4.8 Parser — Análisis Bottom-Up

El **parser** en parser.cpp implementa el algoritmo de parsing **shift-reduce**. Usa una pila y la tabla de acción+goto:

- **SHIFT**: empuja el token actual en la pila y avanza al siguiente estado.
- **REDUCE**: reemplaza los símbolos en la pila que coinciden con el lado derecho de una producción por el no-terminal del lado izquierdo.
- **ACCEPT**: la entrada es válida y se devuelve el árbol sintáctico.
- **ERROR**: el token actual no es válido en el estado actual.

// Traza de parsing shift-reduce

```
// Ejemplo de traza de parsing para '3 + 4' // Gramática: E -> E + T | T, T -> INT //
Pila | Entrada restante | Acción // [0] | INT PLUS INT $ | shift(s2) // [0,2] | PLUS
INT $ | reduce T->INT // [0,T] | PLUS INT $ | goto(s4) // [0,4] | PLUS INT $ | reduce
E->T // [0,E] | PLUS INT $ | goto(s1) // [0,1] | INT $ | shift(s6) // [0,1,6] | $ |
reduce T->INT // [0,1,T] | $ | goto(s7) // [0,1,7] | $ | reduce E->E+T // [0,E] | $ |
goto(s1) // [0,1] | $ | ACCEPT
```

4.9 Resolución de Conflictos

En algunas gramáticas, la tabla puede tener **conflictos**: la misma celda tiene más de una acción posible.

- **Shift/Reduce**: ¿hacer shift o reducir? Ocurre típicamente con la gramática del if-then-else (dangling else).
- **Reduce/Reduce**: dos producciones distintas pueden reducir. Suele indicar ambigüedad real en la gramática.

El módulo conflict_resolver.cpp resuelve automáticamente los conflictos shift/reduce usando las declaraciones de **precedencia y asociatividad** (%left, %right). Si el token tiene mayor precedencia que la producción, se hace shift; si la producción tiene mayor precedencia, se reduce.

5. API REST (Python / FastAPI)

La API es el puente entre el frontend y el núcleo C++. Está construida con **FastAPI**, que genera automáticamente documentación interactiva en /docs.

Endpoints principales:

Endpoint	Método	Descripción
/yalex/analyze	POST	Analiza un .yalex + texto → tokens, DFA, DOT graph
/yapar/analyze	POST	Analiza un .yapar → gramática, autómatas LR(0), tablas SLR/LALR/LL1, árbol
/natural/translate	POST	Traduce texto en Q'eqchi' u otros lenguajes naturales
/graphviz/*	GET	Genera imágenes SVG/PNG desde código DOT (Graphviz)

// Lógica de yalex.py (simplificada)

```
# routers/yalex.py – Endpoint /yalex/analyze @router.post('/analyze') def analyze(req: YAlexRequest): # 1. Escribir archivos temporales (evita problemas con # caracteres especiales en argumentos shell) with tempfile.NamedTemporaryFile(suffix='.yalex') as f: f.write(req.yalex_content) yalex_path = f.name # 2. Ejecutar el binario C++ compilado result = subprocess.run( [YALEX_CLI, yalex_path, input_path], capture_output=True, text=True, timeout=30 ) # 3. Parsear el JSON que devuelve el binario y retornarlo return json.loads(result.stdout)
```

El patrón es siempre el mismo: escribir en archivos temporales → ejecutar binario C++ → parsear JSON de stdout → devolver al frontend. Los archivos temporales se eliminan siempre en el bloque finally.

6. Frontend (React / TypeScript)

El frontend es un **IDE web completo** construido con React 18 y TypeScript. El archivo principal App.tsx tiene ~938 líneas y contiene toda la lógica del IDE.

Componentes principales:

Componente	Archivo	Función
IDE Principal	App.tsx	Coordina editores, estado global, llamadas a la API
Editor de Código	Editor/CodeEditor.tsx	Monaco Editor con syntax highlighting y marcadores de error
Vista de Autómata	Automaton/AutomatonView.tsx	Grafo del autómata LR(0) con Graphviz
Tablas de Parsing	Tables/TableView.tsx	Tablas SLR/LALR/LL1 con celdas coloreadas por tipo de acción
FIRST/FOLLOW	FirstFollow/FirstFollowView.tsx	Conjuntos FIRST y FOLLOW tabulados
Desambiguación	Disambiguation/DisambiguationView.tsx	Detalle de conflictos shift/reduce y su resolución
Árbol Sintáctico	SyntaxTree/TreeView.tsx	Parse tree visual interactivo
Consola	Console/ConsolePanel.tsx	Log de errores y mensajes, clickable para navegar al error

Características destacadas:

- **Detección automática de lenguaje:** el IDE detecta si estás editando .yalex, .yapar, Q'eqchi', MessiScript, etc., y cambia el tema de colores automáticamente.
- **Marcadores de error inline:** los errores del backend se muestran como subrayados rojos en el editor Monaco (igual que VS Code). Hacer clic en el error en la consola salta a la línea exacta.
- **Estado global con Zustand:** todo el estado (archivos, resultados, errores, tema) vive en un store global para evitar prop-drilling.
- **Tres editores:** uno para el .yalex, uno para el .yapar, y uno para el texto de entrada a analizar.
- **Seis pestañas de visualización:** Tokens, Autómata, Tablas, FIRST/FOLLOW, Desambiguación, Traducción.

7. Flujo Completo — Ejemplo Paso a Paso

Imaginemos que queremos analizar la expresión $3 + 4 * 2$ con una gramática de expresiones aritméticas. Esto es lo que ocurre internamente cuando el usuario hace clic en "Ejecutar":

Paso 1 — Frontend envía la petición

El navegador hace POST `/yalex/analyze` con el contenido del `.yalex` y el texto `'3 + 4 * 2'`.

Paso 2 — API recibe y prepara

FastAPI recibe el JSON, escribe el `.yalex` en `/tmp/abc.yalex` y el texto en `/tmp/def.txt`.

Paso 3 — Ejecuta yalex_cli

Invoca: `./core/build/yalex_cli /tmp/abc.yalex /tmp/def.txt`
El binario lee el `.yalex`, construye DFA, y tokeniza el texto.

Paso 4 — Tokens reconocidos

DFA reconoce: `INT(3), PLUS(+), INT(4), TIMES(*), INT(2), EOF($)`
El binario imprime en stdout un JSON con los tokens y el DFA.

Paso 5 — API reenvía al frontend

FastAPI parsea el JSON y lo retorna. El frontend actualiza la pestaña 'Tokens'.

Paso 6 — Análisis sintáctico

Frontend envía POST `/yapar/analyze` con el `.yapar` y los tokens.
`yapar_cli` carga la gramática, computa FIRST/FOLLOW, construye el LR(0), genera tablas.

Paso 7 — Parsing shift-reduce

El parser procesa: `SHIFT(3), REDUCE(T->INT), SHIFT(+), SHIFT(4), REDUCE(T->INT), SHIFT(*), SHIFT(2), REDUCE(T->T*INT), REDUCE(E->E+T), ACCEPT`.
Produce el árbol sintáctico.

Paso 8 — Visualizaciones

El frontend recibe el JSON y actualiza: autómata LR(0), tablas SLR/LALR, conjuntos FIRST/FOLLOW, árbol sintáctico. El usuario puede explorar cada vista.

8. Formatos de Entrada: .yalex y .yapar

Guía de referencia rápida para escribir los archivos de especificación.

Formato .yalex completo:

// Plantilla de archivo .yalex

```
(* Comentarios con (* ... *) o con // *) (* 1. Definición de macros (let) *) let digit
= ['0'-'9'] let upper = ['A'-'Z'] let lower = ['a'-'z'] let letter = upper | lower let
id = letter (letter | digit | '_')* let ws = ' ' | '\t' | '\n' (* 2. Reglas léxicas
(rule tokens = ...) *) rule tokens = digit+ { INT } | id { ID } | '+' { PLUS } | '-' {
MINUS } | '*' { TIMES } | '/' { DIV } | '(' { LPAREN } | ')' { RPAREN } | ws { ignore }
```

Formato .yapar completo:

// Plantilla de archivo .yapar

```
(* Sección de declaraciones *) %token INT ID PLUS MINUS TIMES DIV LPAREN RPAREN %start
program (* Precedencia: más abajo = más alta precedencia *) %left PLUS MINUS %left
TIMES DIV (* Separador OBLIGATORIO *) %% (* Sección de reglas gramaticales *) program :
expr ; expr : expr PLUS term (* suma *) | expr MINUS term (* resta *) | term ; term :
term TIMES factor | term DIV factor | factor ; factor : LPAREN expr RPAREN | INT | ID ;
```

Nota importante: los terminales deben escribirse en MAYÚSCULAS y los no-terminales en minúsculas. El separador %% es obligatorio.

9. Resumen y Conclusiones

Este proyecto implementa desde cero una **cadena completa de compilación**: desde leer una especificación léxica en texto plano hasta producir un árbol sintáctico visualizable en el navegador.

Lo que se implementó:

Análisis Léxico (YALex)

Parsing de .yalex → Expansión de macros y clases de caracteres → Conversión de regex a postfix (Shunting-Yard) → NFA (Thompson) → DFA (Subconjuntos) → DFA mínimo (Hopcroft) → Tokenizador.

Análisis Sintáctico (YAPar)

Parsing de .yapar → Gramática aumentada → FIRST/FOLLOW (punto fijo) → Autómata LR(0) (closure + goto) → Tabla SLR(1) → Tabla LALR(1) → Tabla LL(1) → Parser shift-reduce → Árbol sintáctico.

Resolución de conflictos

Detección de conflictos shift/reduce y reduce/reduce. Resolución automática usando precedencia y asociatividad declaradas con %left/%right/%nonassoc.

API REST

FastAPI con endpoints para YALex, YAPar, traducción natural y Graphviz. Patrón: archivos temporales → subprocess → JSON → frontend.

Frontend IDE

Monaco Editor con marcadores inline, 6 pestañas de visualización, detección automática de lenguaje, temas por lenguaje, consola estructurada.

Tecnologías utilizadas:

Capa	Tecnología	Versión / Detalles
Core C++	C++17 + CMake 3.20+	nlohmann/json para I/O, sin librerías de parsing externas
API	Python 3 + FastAPI	Pydantic, CORS, subprocessos, archivos temporales
Frontend	React 18 + TypeScript	Vite, Monaco Editor, Framer Motion, Zustand, D3.js
Build	CMake + Ninja	Compilación en /core/build, produce yalex_cli y yapar_cli

El proyecto demuestra que es posible implementar un compilador web completo combinando C++ para el rendimiento del núcleo algorítmico, Python para la integración rápida vía API, y React para una experiencia de usuario rica. Cada algoritmo (Thompson, Subconjuntos, Hopcroft, LR(0), FIRST/FOLLOW, SLR, LALR) está implementado desde cero en C++17 sin librerías externas de parsing.